



# Classes and Objects

|             |                                                                                                                                                                                       |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Type        |  Lecture                                                                                           |
| Date        | @December 27, 2021                                                                                                                                                                    |
| Lecture #   | 6                                                                                                                                                                                     |
| Lecture URL | <a href="https://youtu.be/TJC0WhS6FNo">https://youtu.be/TJC0WhS6FNo</a>                                                                                                               |
| Notion URL  | <a href="https://21f1003586.notion.site/Classes-and-Objects-a6d38f8b11b44c269cb11bb0eb209107">https://21f1003586.notion.site/Classes-and-Objects-a6d38f8b11b44c269cb11bb0eb209107</a> |
| Week #      | 1                                                                                                                                                                                     |

## Programming with Objects

- Objects are like abstract datatypes
  - Hidden data with set of public operations
  - All interactions through operations — methods ...
- **Class**
  - Template for a data type
  - How a data is stored
  - How public functions manipulate the data
- **Object**
  - Concrete instance of the above mentioned template
  - Each object maintains its separate copy of local data
  - Invoke methods on objects — Equivalent to “**send a message to the object**”

## Example: 2D points

- A point has coordinates `(x, y)`
  - Each point object stores its own internal values `x` and `y` — these are called instance variables
  - For a point `p`, the local values are `p.x` and `p.y`
  - `self` is a special name referring to the current object — `self.x, self.y`
- When we create an object, we need to set it up
  - Implicitly call a constructor function with a fixed name

- In Python, constructor is called `__init__()`
- Parameters are used to set up internal values
- In Python, the first parameter is always `self`

```
class Point:
    def __init__(self, a=0, b=0):
        self.x = a
        self.y = b
```

## Adding methods to a class

- Translation: shift a point by  $(\Delta x, \Delta y)$ 
  - $(x, y) \mapsto (x + \Delta x, y + \Delta y)$
  - Update instance variables

```
# This will go inside the Point class
def translate(self, dx, dy):
    self.x += dx
    self.y += dy
```

- Distance from the origin
  - $d = \sqrt{x^2 + y^2}$
  - Does not update instance variables
  - state of object is unchanged

```
# This will go inside the Point class
def odistance(self):
    import math
    d = math.sqrt(self.x*self.x + self.y*self.y)
    return d
```

## Changing the internal implementation

- Polar coordinates:  $(r, \theta)$ , not  $(x, y)$ 
  - $r = \sqrt{x^2 + y^2}$
  - $\theta = \tan^{-1}(y/x)$
- Distance from origin is just  $r$

```
import math
class Point:
    def __init__(self, a=0, b=0):
        self.r = math.sqrt(a*a + b*b)
        if a == 0:
            self.theta = math.pi/2
        else:
            self.theta = math.atan(b/a)

    def odistance(self):
        return self.r
```

- Translation
  - Convert  $(r, \theta)$  to  $(x, y)$
  - $x = r \cos \theta, y = r \sin \theta$
  - Recompute  $r, \theta$  from  $(x + \Delta x, y + \Delta y)$
- Interface has not changed
  - User need not be aware whether representation is  $(x, y)$  or  $(r, \theta)$

```
def translate(self, dx, dy):
    x = self.r * math.cos(self.theta)
```

```

y = self.r * math.sin(self.theta)
x += dx
y += dy
self.r = math.sqrt(x*x + y*y)
if x == 0:
    self.theta = math.pi/2
else:
    self.theta = math.atan(y/x)

```

## Abstraction

- Users of our code should not know whether `Point` uses `(x, y)` or `(r, theta)`
  - Interface remains identical, or should remain identical
  - Even constructor is the same
- Python allows direct access to instance variables from outside the class

```
p = Point(5, 7)
```

```
p.x = 4 # Point is now (4, 7)
```

- This defeats the purpose of abstraction
- Changing the internal implementation of `Point` can have impact on other code
  - Usually called breaking changes

```

class Point:
    def __init__(self, a=0, b=0):
        self.x=a
        self.y=b

```

```

class Point:
    def __init__(self, a=0, b=0):
        self.r = math.sqrt(a*a + b*b)
        if a==0:
            self.theta = math.pi/2
        else:
            self.theta = math.atan(b/a)

```

## Subtyping and inheritance

- Define `Square` to be a subtype of `Rectangle`
  - Different constructor
  - Same instance variables

```

class Rectangle:
    def __init__(self, w=0, h=0):
        self.width = w
        self.height = h

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

class Square(Rectangle):
    def __init__(self, s=0):
        self.width = s
        self.height = s

```

- The following code is legal

```
s = Square(5)
```

```
a = s.area()
```

```
p = s.perimeter()
```

- `Square` inherits definitions from `area()` and `perimeter()` from `Rectangle`

## Subtyping and inheritance

- Can change the instance variable in `Square`
  - `self.side`

```

class Rectangle:
    def __init__(self, w=0, h=0):
        self.width = w
        self.height = h

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

class Square(Rectangle):
    def __init__(self, s=0):
        self.side = s

```

- The following code gives a runtime error

```

s = Square(5)
a = s.area()
p = s.perimeter()

```

- `Square` inherits definitions of `area()` and `perimeter()` from `Rectangle`
- But `s.width` and `s.height` have NOT been defined
- Subtype is not forced to be an extension of the parent type

- 
- Subclasses and parent class are usually developed separately
  - Implementor of `Rectangle` changes the instance variables

```

class Rectangle:
    def __init__(self, w=0, h=0):
        self.wd = w
        self.ht = h

    def area(self):
        return self.wd * self.ht

    def perimeter(self):
        return 2 * (self.wd + self.ht)

class Square(Rectangle):
    def __init__(self, s=0):
        self.width = s
        self.height = s

```

- The following code gives a runtime error

```

s = Square(5)
a = s.area()
p = s.perimeter()

```

- `Square` constructor sets `s.width` and `s.height`
- But the instance variable names have changed

- 
- Need a mechanism to hide private implementation details
    - Declare component private or public
  - Working within privacy constraints
    - Instance variables `wd` and `ht` of `Rectangle` are private
    - How can the constructor for `Square` set these private variables?
    - `Square` does (and should) not know the names of the private instance variables
  - Need to have elaborate declarations
    - Type and visibility of variables
  - Static type checking catches errors early

## Summary

- A class is a template describing the instance variables and methods for an abstract datatype

- An object is a concrete instance of a class
- We should separate the public interface from the private implementation
- Hierarchy of class to implement subtyping and inheritance
- A language like Python has no mechanism to enforce privacy etc.
  - Can illegally manipulate private instance variables
  - Can introduce inconsistencies between subtypes and parent types
- Use strong declarations to enforce privacy, types
  - Do not rely on programmer disciplines
  - Catch bugs early through type checking